Practical - 4

Practical Title: JavaScript DOM Manipulation, Event Handling, and UI Interactivity

Problem Definition: Add dynamic UI components such as collapsible FAQ, modal popup, image/content slider, notification banner, hamburger menu, and light/dark theme switcher using JavaScript.

A. Theory

1. How are DOM elements selected and modified?

Answer: DOM (Document Object Model) elements are selected in JavaScript using modern selector methods provided by the document object:

- **document.getElementById('id'):** Selects a single unique element by its ID attribute. It is the fastest selection method.
- **document.querySelector('selector'):** Selects the first element matching any valid CSS selector (e.g., .class, #id, tag[attr]).
- **document.querySelectorAll('selector'):** Returns a static NodeList containing all matching elements, which can be iterated over using .forEach().

Once selected, elements are modified dynamically in several ways:

- **Content Modification:** Using element.textContent or element.innerText to safely update plain text without HTML parsing risks (preventing XSS), or element.innerHTML when inserting structured HTML markup.
- **Attribute Modification:** Using element.setAttribute(name, value), element.getAttribute(name), element.removeAttribute(name), or direct property access (e.g., img.src = 'banner.jpg').
- **Class & Style Manipulation:** Using element.classList.add(), element.classList.remove(), element.classList.toggle(), and element.classList.contains() to trigger CSS classes and transitions cleanly without hardcoding inline styles. Direct styles can also be applied via element.style.propertyName.

Reference: https://developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model/Introduction and https://www.w3schools.com/js/js_htmldom_methods.asp

2. Are event listeners attached correctly?

Answer: Yes, event listeners are attached using the standard element.addEventListener(eventType, handlerFunction, options) method rather than inline HTML event attributes (such as onclick="...").

Using addEventListener provides several critical architectural advantages:

Separation of Concerns: Keeps HTML markup clean and declarative while maintaining all behavioral logic in external JavaScript files.

Multiple Listeners: Allows binding multiple independent listeners to the same event type on a single element without overwriting previous handlers.

Event Flow Control & Delegation: Enables listening during the capturing or bubbling phases.

For dynamically created items or repeated list structures (like FAQ accordions), event delegation is implemented by binding a single listener to a common parent container and inspecting event.target.closest('.item'), drastically reducing memory consumption.

Event Object Handling: Event callbacks properly accept the event object to manage behavior like event.preventDefault() (to stop form reloads or anchor jumps) and event.stopPropagation() (to prevent click bubbling when interacting with modal content).

Reference: https://developer.mozilla.org/en-US/docs/Web/API/EventTarget/addEventListener and https://javascript.info/event-delegation

3. Is localStorage used for remembering theme choice?

Answer: Yes, the browser's localStorage Web API is used to persist the user's selected theme (light mode or dark mode) across browser sessions, page navigations, and tab closures.

The implementation follows a persistent lifecycle:

Checking Existing State on Page Load: When the script loads, it checks localStorage.getItem('studenthub-theme'). If a saved preference exists ('dark' or 'light'), that theme class is applied immediately to the document.body to prevent the flash of unstyled content (FOUC). If no preference is stored, window.matchMedia('(prefers-color-scheme: dark)') is evaluated as a fallback.

Updating and Storing on User Toggle: When the user clicks the theme switcher toggle button, the JavaScript function checks the current active theme, toggles the .dark-mode class on the <body>, and saves the updated state to storage using localStorage.setItem('studenthub-theme', updatedTheme).

Persistence: Because localStorage stores key-value string pairs with no expiration time, the user's customized UI preference remains active indefinitely until the browser storage is cleared.

Reference: https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage and https://web.dev/articles/color-scheme

4. Does interactivity improve usability without breaking accessibility?

Answer: Yes, interactivity has been implemented to enhance user experience while strictly preserving Web Content Accessibility Guidelines (WCAG 2.2) standards:

ARIA Attributes & States: Interactive elements update appropriate ARIA states when manipulated. For example, collapsible FAQ headers dynamically toggle aria-expanded="true/false" and associate with the content panel via aria-controls. Modals use role="dialog", aria-modal="true", and aria-labelledby.

Keyboard Navigation & Focus Management: All interactive elements (accordion triggers, slider buttons, modal close buttons, theme toggles, hamburger menus) are built using native semantic <button> elements that are focusable via the Tab key. When a modal opens, focus is trapped inside the dialog and transferred to the close button, and pressing the Escape key closes active modals or menus.

Screen Reader Announcements: Notification banners include role="alert" or aria-live="polite" so screen readers immediately announce system updates without interrupting the user.

Motion Preferences: CSS transitions and animations applied during slide/collapse transitions are scoped with @media (prefers-reduced-motion: reduce) to support users sensitive to motion sickness or vestibular disorders.

Reference: https://www.w3.org/WAI/ARIA/apg/ and https://web.dev/learn/accessibility/

B. Implementation/ Output/Result

The interactive UI components were implemented across the StudentHub portal using vanilla JavaScript (ES6+), semantic HTML5, and CSS3 transitions:

Collapsible FAQ Accordion: Interactive questions using event delegation on parent containers to smoothly toggle question answers with active height transitions and icon state rotation.

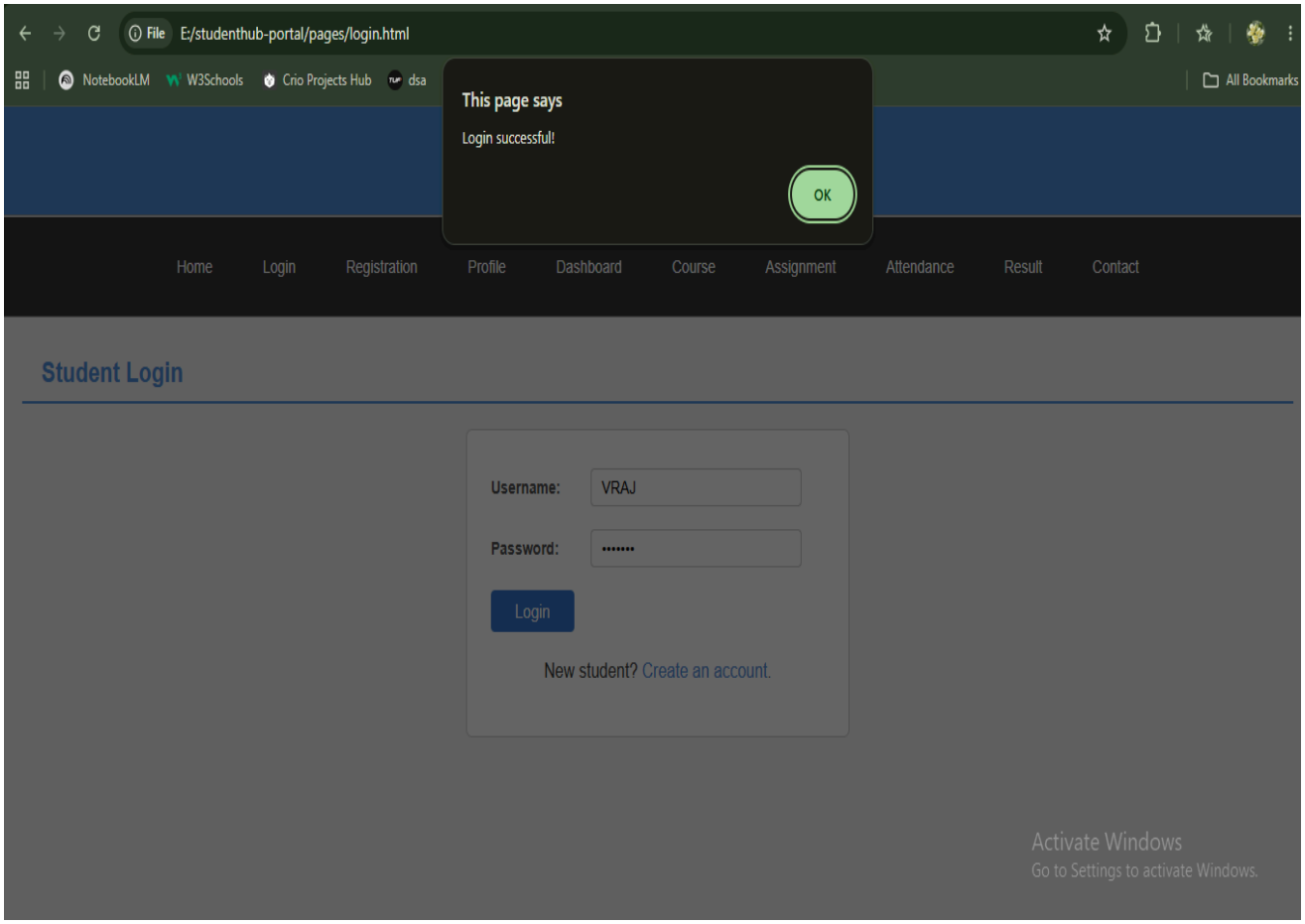
Modal Popup: Accessible inquiry dialog with backdrop overlay blur, focus management, outside-click detection, and Escape key listeners.

Image / Content Carousel Slider: Touch and click supported multi-card carousel with automatic cycling, previous/next controls, and active indicator dot synchronization.

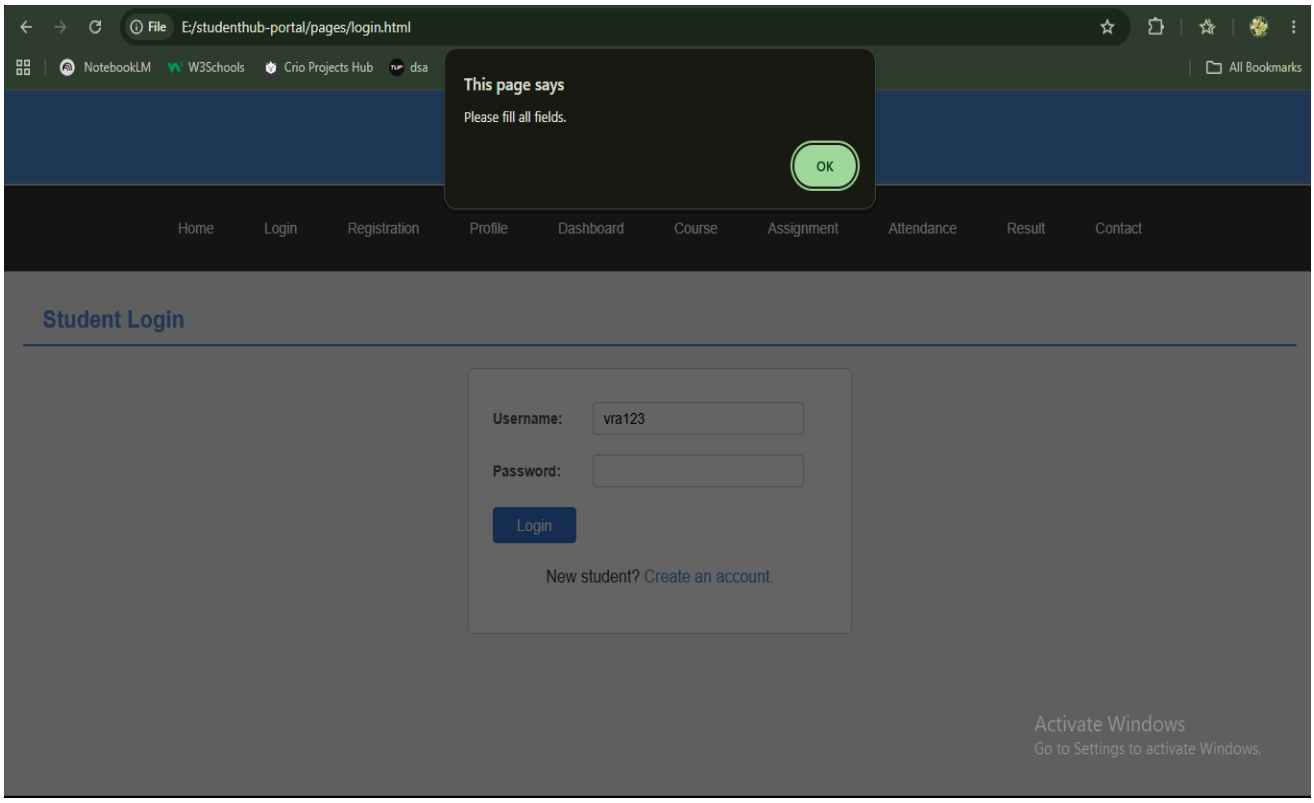
Notification Banner: Top announcement banner with dismiss button and session-based dismissal tracking.

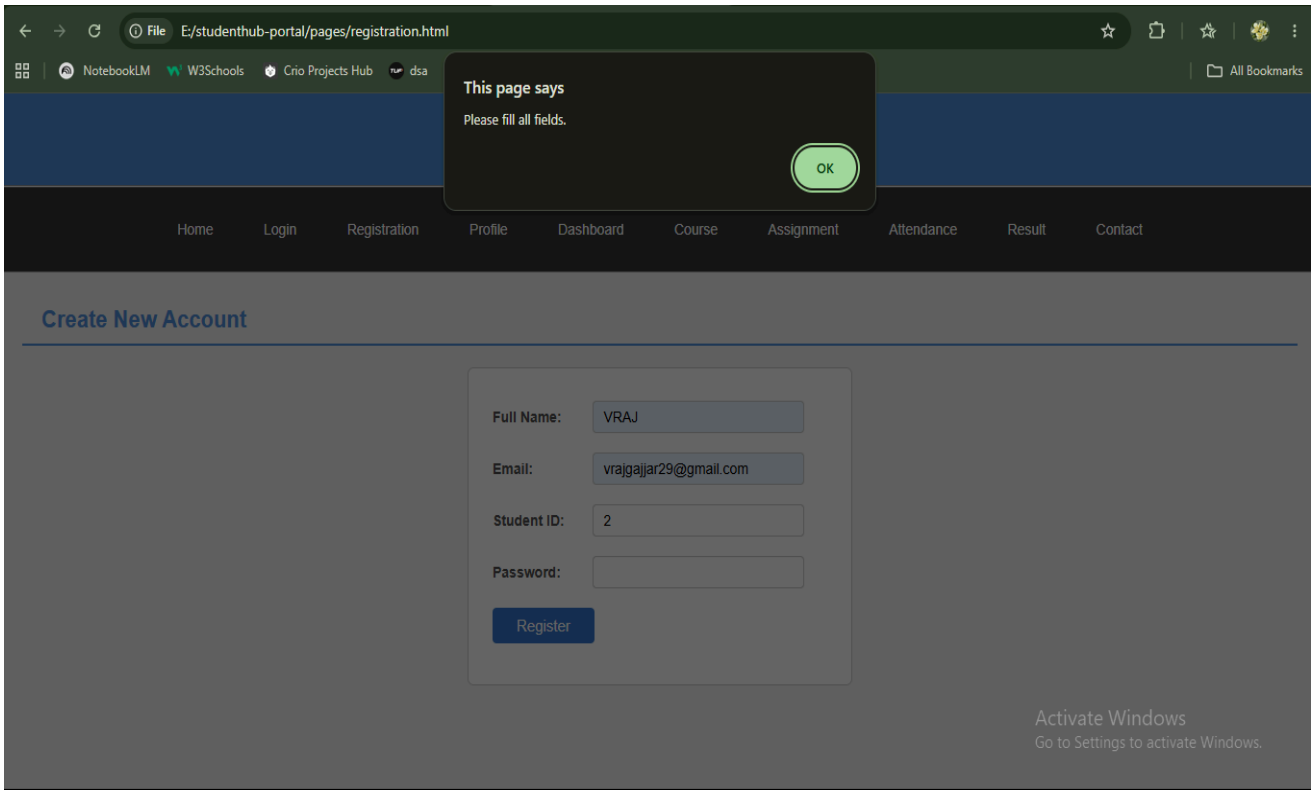
Responsive Hamburger Navigation: Mobile breakpoint dropdown menu triggered via animated hamburger toggle with aria-expanded state synchronization.

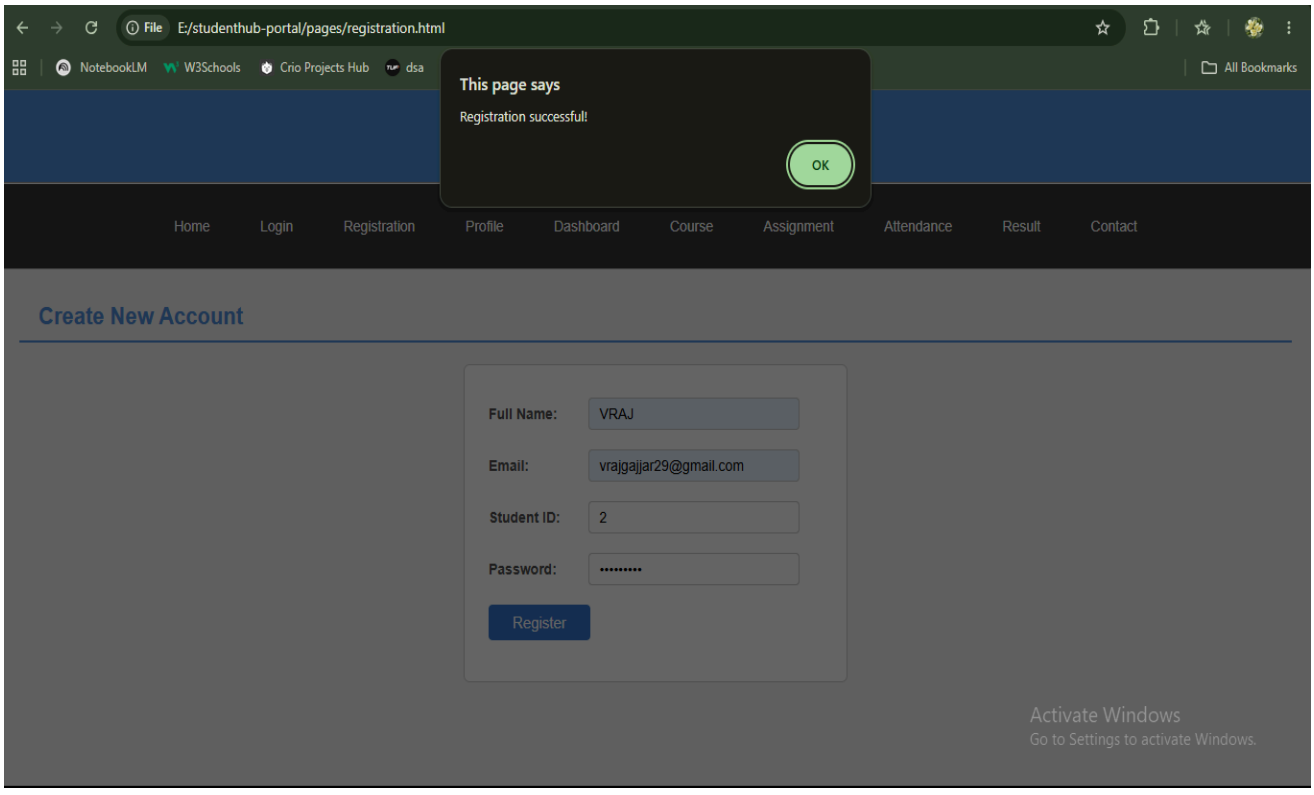
Light / Dark Mode Theme Switcher: Dynamic CSS variable-driven theme switcher with instant state toggling and persistent browser storage via localStorage.



C. Evidence of Work (Screenshots)


File E:/studenthub-portal/pages/login.html
NotebookLM
W3Schools
Crio Projects Hub
dsa
All Bookmarks
This page says
Please fill all fields.
OK
Home
Login
Registration
Profile
Dashboard
Course
Assignment
Attendance
Result
Contact
Student Login
Username:
vra123
Password:
Login
New student? Create an account.
Activate Windows
Go to Settings to activate Windows.


File E:/studenthub-portal/pages/registration.html
NotebookLM W3Schools Crio Projects Hub dsa
All Bookmarks
This page says
Please fill all fields.
OK
Home Login Registration Profile Dashboard Course Assignment Attendance Result Contact
Create New Account
Full Name: VRAJ
Email: vrajgajjar29@gmail.com
Student ID: 2
Password:
Register
Activate Windows
Go to Settings to activate Windows.


File E:/studenthub-portal/pages/registration.html
NotebookLM W3Schools Crio Projects Hub dsa
All Bookmarks
This page says
Registration successful!
OK
Home Login Registration Profile Dashboard Course Assignment Attendance Result Contact
Create New Account
Full Name: VRAJ
Email: vrajgajjar29@gmail.com
Student ID: 2
Password: ••••••••
Register
Activate Windows
Go to Settings to activate Windows.

D. GitHub Repository Link:

https://github.com/VRAJ-710/studenthub-portal

E. Challenges Faced:

1. Managing State and Event Bubbling: In the collapsible FAQ accordion, clicking on inner elements (such as tags inside the button) initially broke element targeting until event.target.closest('.faq-question') was implemented for event delegation.

2. Preventing Flash of Unstyled Content (FOUC) on Theme Load: Loading theme settings after full DOM render caused a brief white flicker in dark mode. This was solved by reading localStorage immediately during script initialization to set body classes prior to element rendering.

3. Focus Trapping & Accessible Dialogs: Ensuring that pressing the Tab key does not escape the inquiry modal into hidden background links while open, and cleanly wiring the Escape key listener for instant closing.

4. Boundary Checks in Carousel Slider: Handling negative index transitions when clicking the 'Previous' button on the first slide, resolved using modular arithmetic (index + totalSlides) % totalSlides.

F. Learning Outcomes:

From this practical, I learned how to utilize JavaScript ES6+ DOM manipulation and event listener APIs to create rich, dynamic, and responsive web user interfaces without external frontend frameworks. I gained hands-on experience with:

Selecting and altering DOM elements using querySelector, classList, and property bindings.

Implementing robust event handling with addEventListener, event delegation, and controlling default behaviors via event.preventDefault().

Persisting client-side UI configurations across browser sessions using the localStorage and sessionStorage Web APIs.

Designing accessible UI widgets (modals, accordions, hamburger menus) that comply with ARIA states and keyboard accessibility rules (WCAG).

Combining JavaScript state toggles with smooth CSS transitions for seamless micro-interactions.

Faculty Remarks: